2- Air Traffic Control Sorting Problem

Merge Sort algorithm.

Justification for Choosing Merge Sort:

1. **Stability:** Merge Sort is a stable sorting algorithm. This means that if two flights have the same sorting criterion (e.g., the same arrival time), their original relative order in the input list will be preserved in the sorted output. This is important in air traffic control as maintaining the original order of flights with the same priority can be beneficial for various operational reasons.
2. **Guaranteed Time Complexity:** Merge Sort has a time complexity of O(nlogn) in the worst, average, and best cases. This consistent performance is crucial in real-time systems like air traffic control where predictable execution times are essential. Unlike algorithms like Quick Sort which can degrade to O(n2) in the worst case, Merge Sort provides a reliable upper bound on the sorting time.
3. **Suitability for Large Datasets:** Air traffic control systems deal with a large number of flights. Merge Sort performs well on large datasets due to its divide-and-conquer nature.
4. **Parallelizability:** Merge Sort can be easily parallelized, which can further improve its performance in a multi-processor environment, potentially found in advanced air traffic control systems.

Implementation (Python):

```
class Flight:
    def __init__(self, flight_id, time, altitude, speed):
        self.flight_id = flight_id
        self.time = time
        self.altitude = altitude
        self.speed = speed
    def __repr__(self):
        return f"Flight(ID='{self.flight_id}', Time={self.time}, Alt={self.altitude}, Speed={self.speed})"
def merge(left, right):
    merged = []
    left_index = 0
    right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index].time <= right[right_index].time:
            merged.append(left[left_index])
            left_index += 1
```

```
        else:
            merged.append(right[right_index])
            right_index += 1
    merged.extend(left[left_index:])
    merged.extend(right[right_index:])
    return merged
def merge_sort_flights(flights):
    if len(flights) <= 1:
        return flights
    mid = len(flights) // 2
    left = flights[:mid]
    right = flights[mid:]
    return merge(merge_sort_flights(left), merge_sort_flights(right))

# Sample Flight Data
flights_data = [
    Flight("BA249", 1000, 35000, 550),
    Flight("EK305", 945, 38000, 580),
    Flight("AA101", 1015, 32000, 520),
    Flight("LH400", 945, 33000, 560),
    Flight("UA200", 1030, 36000, 570),
    Flight("AF118", 1000, 34000, 540),
]
# Sort flights by arrival/departure time
sorted_flights = merge_sort_flights(flights_data)
print("Original Flight Data:")
for flight in flights_data:
    print(flight)
print("\nSorted Flight Data (by Time):")
for flight in sorted_flights:
    print(flight)
```

Testing the Solution:

The sample flight data is created as a list of Flight objects. The merge_sort_flights function is called to sort this list based on the time attribute. The output shows the original and sorted flight data, demonstrating that the flights are now ordered according to their arrival/departure times. Notice that the two flights with the same time (BA249 and AF118 at 1000) maintain their relative order in the sorted list, illustrating the stability of Merge Sort.

Impact on Real-Time Air Traffic Management in a High-Traffic Scenario.